As Agreed

An agent that pays supplier invoices only as far as they match what was agreed.

Concept for the Airwallex Agentic Banking Hackathon · Starter kits 1 (Adaptive Treasury Controller) and 3 (Payment Ops Incident Commander), with reconciliation · 20 October 2026 · No code yet: the build starts on 25 October.

THE PROBLEM

A finance team paying supplier invoices in several currencies rarely loses money on arithmetic. It loses it on the payment that should never have left:

- a duplicate invoice
- a line nobody ordered
- an email saying "we changed our bank details"
- a transfer that fails quietly and is then paid twice
- an amount that drifts after FX and fees

THE CHECK: EVERY INVOICE LINE, BEFORE ANY MONEY MOVES

What was agreed
purchase orders and contracts

Who may be paid
the verified beneficiary record

The payment policy, in code
autonomous limit, duplicate lock, reserve floor, FX tolerance

→

matched
an agreed line supports it, no rule blocks it, the beneficiary is the verified one
paid through Airwallex within the autonomous limit

conflict
a rule blocks it: duplicate, changed beneficiary, over the limit, below the reserve floor
payment halts; a person decides

proposal
nothing that was agreed supports it
shown to the approver; never paid without approval

uncertain conversion
after FX and fees the payable amount drifts beyond tolerance
goes to the approver

Each line has exactly one state. Precedence: conflict > proposal > uncertain conversion > matched. A halt a rule fired stays a halt, whatever a model says later.

THE DEMO

1 Goal
Pay this week's six supplier invoices: USD, GBP and EUR, four suppliers.

2 Initial plan
A state for every line; FX, the SWIFT fee and the reserve floor priced in.

3 New information
A supplier emails new bank details. One transfer ends `CANCELLED` with a failure type.

4 Revised decision
That supplier is held and verified out of band. The failed transfer is replaced under a duplicate lock; the original's fate is recorded.

5 Result
Balances, payments and holds, and a reconciliation report: every paid line traces to its invoice, agreement and approval.

BUILT ON AIRWALLEX (SANDBOX)

- **Plan:** `balances/current`, `fx/rates/current`, `fx/quotes/create`, `fx/conversions/create`
- **Pay:** `beneficiaries/schema`, `/validate`, `/create`; `transfers/create`; US and GB local, EUR over SWIFT
- **Stage:** `simulation/deposit/create`, `simulation/transfers/{id}/transition`, behind one interface
- **Reconcile:** `GET transfers` and the payout webhooks (paid, failed, cancelled), signature checked and deduplicated

WHO DECIDES WHAT

Code	amounts, limits, the reserve floor, duplicate protection, state transitions, every Airwallex call (typed tools, a new request id per operation)
Models	read invoices, contracts and emails, match lines, explain exceptions; a second model from a different family tries to refute every match
A person	every conflict, proposal and uncertain conversion; each approval bound to the amount, currency, beneficiary and the evidence shown

MEASURED, NOT CLAIMED

no double payment
tested in code; one violation fails the build

no payment to an unverified beneficiary
tested in code

nothing paid below the reserve floor
tested in code

A pre-registered error-injection run (duplicate invoice, wrong amount, changed beneficiary, unordered line, wrong currency, omitted fee, and clean lines) reports recall per error type, false alarms on clean lines and time to decision, with the thresholds locked before the first run and the results reported as they fall.

Previous work. The same verification principle, first built for physiotherapy home programs: As Prescribed, AMD Developer Hackathon ACT III, with its pre-registered evaluation published as it fell (github.com/Sunrisesoftware-app/as-prescribed).

Sunrise Software, Tampere, Finland · github.com/Sunrisesoftware-app/as-agreed · Synthetic data and the Airwallex sandbox only; no real company, person or payment data.